

● Verilog Coding & Digital Design Fundamentals

1. Verilog Syntax & Structure

- Module definition
- Port declaration (input, output, inout)
- Positional vs Named port mapping
- Continuous assignment (assign)
- Procedural blocks (always, initial)

2. Common Verilog Constructs

- reg vs wire
- always @(*) for combinational logic
- always @(posedge clk) for sequential logic
- parameter usage for module parameterization
- case, if-else statements in procedural code

3. Flip-Flops and Latches

- Positive and negative edge-triggered D flip-flops
- Asynchronous and synchronous reset/set
- Flip-flop timing parameters (setup time, hold time, clock-to-Q delay)
- Flip-flop metastability: causes and behavior

4. State Machines (FSMs)

- Mealy vs Moore machines
- State assignment using parameter
- Next state logic
- Output logic
- Minimum number of flip-flops required
- Implementing FSMs in Verilog

5. Memory and Data Storage

- Difference between SRAM and DRAM
 - Defining memory arrays in Verilog
 - reg arrays for memory implementation
 - Tristate buffers and their use in bidirectional buses
-

● Digital System Design Concepts

6. Timing Analysis & Constraints

- Setup time and hold time calculation
- Slack time interpretation (WNS in Vivado)
- Maximum clock frequency determination

7. Clocking Techniques

- Phase-Locked Loop (PLL) functions and applications
- Clock recovery and jitter reduction

8. Pipelining

- Impact on latency and throughput
- Register use in pipelining

9. Communication Concepts

- Serial communication modes (Simplex, Half-duplex, Full-duplex)
- Baud rate definition
- Differential pairs and noise immunity
- XADC usage in FPGAs

10. Error Detection

- Parity bit usage
 - Single-bit error detection
-

⚙ Verilog Practical Implementation

11. Module Instantiation

- Positional and named port mapping

12. Structural vs Behavioral Coding

- When to use each
- Declaration differences (reg vs wire)

13. Tristate Buffers

- Purpose and functionality